

claude-windows / 6e9d0958-a2ec-4a8e-bb2d-6cce9833c527

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\subagents\agent-acf8481876a7ec19a.jsonl`  
- SHA-256: `709c780de05408f4f5a12246d5e05148731c96618892f07e11254bcb8ad9d23b`  
- Source modified: `2026-07-04T00:45:10+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `subagents`  
- session_id: `6e9d0958-a2ec-4a8e-bb2d-6cce9833c527`
```

Transcript

1. user (2026-07-04T00:44:43.482Z)

Code-review finder. Target: PR #458 of badminton-highlight-indexer. Diff: `cd C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer && gh pr diff 458`. PR branch = `origin/codex/a7-a9-quality-chain`. Files at scratchpad/pr458_fusion_golden.py, pr458_fusion_audio.py.

ANGLE ? simplification. Flag unnecessary complexity; name the simpler form:

1. The `--missing-fusion-only` flag + `missing\_fusion\_only` param in run(). Trace the resume branches carefully:

```
- `if missing_fusion_only and not fresh and existing_full:` ? skip+continue  
- `if not fresh and existing and existing_full:` ? skip+continue  
- `if not fresh and existing and not existing_full:` ? print "re-extracting" (falls through to extract)
```

Both the 1st and 2nd branch skip when existing_full. So when existing_full is True, the video is skipped REGARDLESS of missing_fusion_only ? the flag changes only a log message. And the DEFAULT (no flag) already re-extracts windowing-only and skips full-fusion. So does `--missing-fusion-only` actually DO anything different from the default? If it's a no-op, that's dead complexity + a misleading CLI flag. State precisely what (if anything) the flag changes.

2. Duplicated source-manifest attach block: the SAME ~6-line block (`source\_manifest = source\_manifest or os.environ.get(SOURCE\_MANIFEST\_ENV); if source\_manifest: updated = attach\_source\_video\_paths(...); print(...)`) appears in BOTH fusion_golden.run() and fusion_audio.augment(). Could be one helper.

3. `load\_source\_video\_paths` handling of `data.get("videos", data)` then the isinstance list checks ? any redundant/dead branch?

4. The `\_progress` closure now returns/uses `rec` ? is the nesting/logic clean or could the temp-check be its own small function?

Only flag where the simpler form is nameable. Return up to 6 candidates as JSON [{file, line, summary, failure_scenario}]. [] if none. Final message = only the JSON array.

2. assistant (2026-07-04T00:44:45.483Z)

I'll analyze the PR diff for simplification opportunities, focusing on the specific angles you mentioned. Let me start by examining the diff and the provided files.

3. user (2026-07-04T00:44:47.474Z)

<persisted-output>

Output too large (51.9KB). Full output saved to: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\tool-results\bagfasnt2.txt

Preview (first 2KB):

```
diff --git a/SESSION_HANDOFF.md b/SESSION_HANDOFF.md  
index 31ea4c14..a50cc8a1 100644  
--- a/SESSION_HANDOFF.md  
+++ b/SESSION_HANDOFF.md
```

@@ -2,49 +2,51 @@

You are here

-A6 is implemented and open for review as PR #455:

-`https://github.com/Khelsutra/badminton-highlight-indexer/pull/455`.

+A6 PR #455 is merged. This checkout is on `codex/a7-a9-quality-chain`, fast-forwarded to
+`origin/master` at `c5fc27c`, with tracked edits in:

-The branch is `codex/a6-ablation-cli`, rebased onto `origin/master` after A5

-merged as `c8eec7c` / PR #454. It fixes the `python -m backend.eval.ablation`

-circular import by making `ablation.export\_pdf` a lazy wrapper, adds a

-module-entrypoint regression test, and records the n=15 R2/tolF1 contrast in

-`docs/ALPHA\_LAUNCH\_READINESS.md`.

+`- backend/eval/fusion\_golden.py`

+`- backend/eval/fusion\_audio.py`

+`- tests/test\_fusion\_compare.py`

+`- tests/test\_audio\_features.py`

+`- docs/ALPHA\_LAUNCH\_READINESS.md`

+`- SESSION\_HANDOFF.md`

-Validation completed locally on 2026-07-03:

-`ruff check .`, `python -m mypy backend training`, and

-`python run\_all\_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70`

-all passed. Full suite result: 1609 passed, 6 skipped, 85.65% coverage.

+A7/A8/A9 have been run locally against the 15-video corpus, and draft PR #458 is open:

+`https://github.com/Khelsutra/badminton-highlight-indexer/pull/458`. The generated

+feature/model artifacts live under ignored `output/` and `artifacts/` paths and should not
+be committed.

Next steps / open threads

-1. Review/merge PR #455 for A6.

-2. A7 is the next indexer-owned quality item: regenerate or upconvert

- full-fusion feature JSONs for the newer nine golden videos so fusion/person

- /audio analyses can run on all 15 videos. GPU is approved on this machine.

-3. A8 follows A7: rerun `fusion\_compare`, group ablation, and served contrast;

- explicitly promote or reject fusion/person/Gate-A served-default decisions.

...

</persisted-output>

4. assistant (2026-07-04T00:44:49.246Z)

Now let me read the full diff and the provided files:

5. user (2026-07-04T00:44:51.083Z)

1 diff --git a/SESSION_HANDOFF.md b/SESSION_HANDOFF.md

2 index 31ea4c14..a50cc8a1 100644

3 --- a/SESSION_HANDOFF.md

4 +++ b/SESSION_HANDOFF.md

5 @@ -2,49 +2,51 @@

6

7 ## You are here

8

9 -A6 is implemented and open for review as PR #455:

10 -`https://github.com/Khelsutra/badminton-highlight-indexer/pull/455`.

11 +A6 PR #455 is merged. This checkout is on `codex/a7-a9-quality-chain`, fast-forwarded
to

12 +`origin/master` at `c5fc27c`, with tracked edits in:

13

14 -The branch is `codex/a6-ablation-cli`, rebased onto `origin/master` after A5

15 -merged as `c8eec7c` / PR #454. It fixes the `python -m backend.eval.ablation`

16 -circular import by making `ablation.export\_pdf` a lazy wrapper, adds a

17 -module-entrypoint regression test, and records the n=15 R2/tolF1 contrast in

```

18 - `docs/ALPHA_LAUNCH_READINESS.md`.
19 +- `backend/eval/fusion_golden.py`
20 +- `backend/eval/fusion_audio.py`
21 +- `tests/test_fusion_compare.py`
22 +- `tests/test_audio_features.py`
23 +- `docs/ALPHA_LAUNCH_READINESS.md`
24 +- `SESSION_HANDOFF.md`
25
26 -Validation completed locally on 2026-07-03:
27 -`ruff check .`, `python -m mypy backend training`, and
28 -`python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-
under=70`
29 -all passed. Full suite result: 1609 passed, 6 skipped, 85.65% coverage.
30 +A7/A8/A9 have been run locally against the 15-video corpus, and draft PR #458 is open:
31 +`https://github.com/Khelsutra/badminton-highlight-indexer/pull/458`. The generated
32 +feature/model artifacts live under ignored `output/` and `artifacts/` paths and should
not
33 +be committed.
34
35 ## Next steps / open threads
36
37 -1. Review/merge PR #455 for A6.
38 -2. A7 is the next indexer-owned quality item: regenerate or upconvert
39 - full-fusion feature JSONs for the newer nine golden videos so fusion/person
40 - /audio analyses can run on all 15 videos. GPU is approved on this machine.
41 -3. A8 follows A7: rerun `fusion_compare`, group ablation, and served contrast;
42 - explicitly promote or reject fusion/person/Gate-A served-default decisions.
43 -4. A3 is owned by another agent; do not duplicate that collector/vault work.
44 +1. Address review comments on draft PR #458; convert to ready only after owner/reviewer
agrees
45 + the A9 TemporalMaxer scope question is settled.
46 +2. Decide whether A9 is considered complete with the current Gen-0 non-serving shadow
47 + bundle or whether TemporalMaxer needs a follow-up scorer-swap PR.
48 +3. Do not touch A1; another agent owns it.
49 +4. A3 is in collector PR #62 and records all 15 source videos; the two Boxhill
originals
50 + were verified on `F:\`.
51
52 ## Ramp-up kit
53
54 - `docs/ALPHA_LAUNCH_READINESS.md`
55 -- `docs/data_pipeline/GOLDEN_VIDEOS.md`
56 -- `docs/R2_EVAL_METRIC_DESIGN.md`
57 -- `backend/eval/ablation.py`
58 -- `backend/eval/ablation_pdf.py`
59 -- `backend/eval/rally_seg_eval.py`
60 -- `tests/test_ablation.py`
61 -- `output/human_lovo_manifest.json`
62 +- `backend/eval/fusion_golden.py`
63 +- `backend/eval/fusion_audio.py`
64 +- `training/gen0/harness.py`
65 +- `eval_baselines/heuristic_lovo_n15.json`
66 +- `output/a7_remaining_default.out.log`
67 +- `output/a8_audio_augment.out.log`
68 +- `output/a6_r2_ablation.json`
69 +- `artifacts/rally_tal_gen0/0.1.1-n15-shadow.20260704/manifest.json`
70
71 ## Key decisions
72
73 -- R2/tolF1 is supported for the baseline-off to served milestone contrast on
74 - the 15-video corpus: `Delta tolF1 = +0.225`, CI `[+0.165, +0.283]`,
75 - sign `14/0`, `p = 0.000`.
76 -- This does not approve fusion/person/audio default changes. Current full-fusion
77 - ablation still runs on only six videos because the newer nine do not yet have
78 - full-fusion feature schema.

```

```

79 -- Branching discipline remains important: always fetch/pull first, branch from
80 - `origin/master`, stage explicit paths, and verify `origin/master..HEAD` before
81 - pushing.
82 +- A7: full-fusion schema and audio features are present locally for all 15 videos.
83 +- A8: person fusion and audio are suggestive but not statistically clean; do not
promote
84 + either as a default. Gate A regresses under served windowing; keep it opt-
in/analysis-only.
85 +- R2/cap/inactivity remains the strongest quality result on n15: F1 `0.237 -> 0.472`,
86 + tolF1 `0.138 -> 0.364`, merge rate `0.638 -> 0.135`; net over-seg guard passes.
87 +- A9: Gen-0 LogReg/TAL n15 shadow bundle beats the heuristic mean (`0.551 > 0.4456`)
88 + but is non-serving (`9/15` paired wins, `p=0.3036`; #172 and WASB C3 still block).
89 +- Branching discipline remains important: fetch/pull first, stage explicit paths, and
90 + verify `origin/master..HEAD` contains only this branch's commits before pushing.
91 diff --git a/backend/eval/fusion_audio.py b/backend/eval/fusion_audio.py
92 index b5c65ec4..7dff0c94 100644
93 --- a/backend/eval/fusion_audio.py
94 +++ b/backend/eval/fusion_audio.py
95 @@ -17,14 +17,17 @@
96     import json
97     import logging
98     import os
99 -from typing import Any, Dict, List
100 +from typing import Any, Dict, List, Optional, Sequence
101
102     from backend.eval import experiment
103     from backend.eval.fusion_compare import delta_stats
104     from backend.eval.fusion_golden import (
105         PERSON_FEATURE_NAMES,
106         SHUTTLE_FEATURE_NAMES,
107     + SOURCE_MANIFEST_ENV,
108         _derive_video_path,
109     + _filter_specs,
110     + attach_source_video_paths,
111     )
112     from backend.eval.golden_manifest import load_resolved_manifest
113     from backend.pipeline.audio.hit_detector import detect_hits_in_video
114 @@ -33,9 +36,24 @@
115     logger = logging.getLogger(__name__)
116
117
118     -def augment(manifest_path: str, features_dir: str, k: float = 2.5) -> List[str]:
119     +def augment(
120     +     manifest_path: str,
121     +     features_dir: str,
122     +     k: float = 2.5,
123     +     source_manifest: Optional[str] = None,
124     +     only: Sequence[str] = (),
125     +) -> List[str]:
126         """Add the 3 audio features to each existing ``<name>_golden_features.json``
window."""
127         specs = load_resolved_manifest(manifest_path)
128         source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
129         if source_manifest:
130             updated = attach_source_video_paths(specs, source_manifest)
131             print(
132             +         f"[fusion_audio] attached {updated}/{len(specs)} source video path(s) "
133             +         f"from {source_manifest}",
134             +         flush=True,
135             +     )
136             specs = _filter_specs(specs, only)
137             written: List[str] = []
138             for spec in specs:
139                 jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
140 @@ -45,6 +63,10 @@ def augment(manifest_path: str, features_dir: str, k: float = 2.5) ->

```

```

List[str]:
141         )
142         continue
143         video = _derive_video_path(spec)
144         +         if not os.path.isfile(video):
145         +             raise FileNotFoundError(
146         +                 f"{spec.get('name')}: source video not found for audio augmentation:
{video}"
147         +             )
148         print(
149             f"[fusion_audio] {spec['name']}: detecting audio hits (k={k})...",
150             flush=True,
151         @@ -129,6 +151,21 @@ def main() -> None:
152             ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
153             ap.add_argument("--features-dir", default="output")
154             ap.add_argument("--k", type=float, default=2.5, help="hit-detector sensitivity")
155         +         ap.add_argument(
156         +             "--source-manifest",
157         +             default=None,
158         +             help=(
159         +                 "collector golden_source_videos.json; overrides video paths for
source/proxy "
160         +                 f"videos (or set {SOURCE_MANIFEST_ENV})"
161         +             ),
162         +         )
163         +         ap.add_argument(
164         +             "--only",
165         +             action="append",
166         +             default=[],
167         +             metavar="VIDEO_ID",
168         +             help="augment one manifest video by name; repeatable",
169         +         )
170         ap.add_argument(
171             "--augment",
172             action="store_true",
173         @@ -142,7 +179,13 @@ def main() -> None:
174             args = ap.parse_args()
175
176             if args.augment:
177                 -             augment(args.manifest, args.features_dir, args.k)
178                 +             augment(
179                 +                 args.manifest,
180                 +                 args.features_dir,
181                 +                 args.k,
182                 +                 source_manifest=args.source_manifest,
183                 +                 only=args.only,
184                 +             )
185             if args.compare:
186                 videos = load_videos_with_audio(args.features_dir)
187                 if len(videos) < 2:
188                 diff --git a/backend/eval/fusion_golden.py b/backend/eval/fusion_golden.py
189                 index aee6d7e5..0666a0ca 100644
190                 --- a/backend/eval/fusion_golden.py
191                 +++ b/backend/eval/fusion_golden.py
192                 @@ -21,7 +21,7 @@
193                 import json
194                 import os
195                 import time
196                 -from typing import Any, Dict, List, Optional
197                 +from typing import Any, Dict, List, Optional, Sequence
198
199                 from backend.eval import distill_local
200                 from backend.eval.calibrate_wasb import load_golden_gts
201                 @@ -36,6 +36,100 @@
202                 SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)

```

```

203     # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
204     PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
205 +SOURCE_MANIFEST_ENV = "RALLY_GOLDEN_SOURCE_MANIFEST"
206 +
207 +
208 +def is_full_fusion_feature_file(path: str) -> bool:
209 +     """True when an existing golden feature JSON already has shuttle+person cue dicts.
210 +
211 +     Corpus-growth runs can leave leaner windowing-only JSONs with the same filename
212 +     pattern.
213 +     Those are useful for gts/windowing analysis, but they are not the replayable fusion
214 +     substrate that ``fusion_compare`` and ``ablation`` need.
215 +
216 +     """
217 +     try:
218 +         with open(path, encoding="utf-8") as f:
219 +             data = json.load(f)
220 +     except (OSError, json.JSONDecodeError, TypeError):
221 +         return False
222 +     candidates = data.get("candidates")
223 +     if not isinstance(candidates, list):
224 +         return False
225 +     if not candidates:
226 +         return True
227 +     return all(
228 +         isinstance(candidate, dict)
229 +         and "shuttle" in candidate
230 +         and "person" in candidate
231 +         for candidate in candidates
232 +     )
233 +
234 +def load_source_video_paths(source_manifest_path: Optional[str]) -> Dict[str, str]:
235 +     """Load ``video_id -> local_path`` from the collector golden-source manifest.
236 +
237 +     The collector records source/proxy ownership and full-file MD5s. Only entries
238 +     marked
239 +     ``present_local`` with a concrete ``local_path`` can drive local GPU extraction.
240 +
241 +     """
242 +     if not source_manifest_path:
243 +         return {}
244 +     with open(source_manifest_path, encoding="utf-8") as f:
245 +         data = json.load(f)
246 +     entries = data.get("videos", data) if isinstance(data, dict) else data
247 +     if not isinstance(entries, list):
248 +         raise ValueError(f"source manifest must contain a videos list: {source_manifest_path}")
249 +
250 +     paths: Dict[str, str] = {}
251 +     for entry in entries:
252 +         if not isinstance(entry, dict):
253 +             continue
254 +         video_id = entry.get("video_id") or entry.get("name")
255 +         local_path = entry.get("local_path")
256 +         if not video_id or not local_path:
257 +             continue
258 +         if entry.get("source_status", "present_local") != "present_local":
259 +             continue
260 +         paths[str(video_id)] = str(local_path)
261 +     return paths
262 +
263 +def attach_source_video_paths(
264 +     specs: Sequence[Dict[str, Any]], source_manifest_path: Optional[str]

```

```

265     +) -> int:
266     +     """Attach explicit ``video_path`` fields to manifest specs from the source
manifest.
267     +
268     +     Returns the number of specs updated. The collector source manifest is authoritative
269     +     for local source/proxy video location and overrides derived/stale manifest paths.
270     +     """
271     +
272     +     source_paths = load_source_video_paths(source_manifest_path)
273     +     updated = 0
274     +     for spec in specs:
275     +         name = str(spec.get("name", ""))
276     +         source_path = source_paths.get(name)
277     +         if not source_path:
278     +             continue
279     +         if spec.get("video_path") == source_path:
280     +             continue
281     +         spec["video_path"] = source_path
282     +         updated += 1
283     +     return updated
284     +
285     +
286     +def _filter_specs(
287     +     specs: Sequence[Dict[str, Any]], only: Sequence[str]
288     +) -> List[Dict[str, Any]]:
289     +     if not only:
290     +         return list(specs)
291     +     wanted = list(dict.fromkeys(only))
292     +     wanted_set = set(wanted)
293     +     filtered = [spec for spec in specs if str(spec.get("name")) in wanted_set]
294     +     found = {str(spec.get("name")) for spec in filtered}
295     +     missing = [name for name in wanted if name not in found]
296     +     if missing:
297     +         raise ValueError(f"--only requested unknown video(s): {' ', '.join(missing)}")
298     +     return filtered
299
300
301     def _derive_video_path(spec: Dict[str, Any]) -> str:
302     @@ -58,6 +152,9 @@ def extract_video(
303         iou: float = 0.5,
304         log_every_sec: float = 15.0,
305         telemetry: Optional[Any] = None,
306     +     use_gpu: bool = True,
307     +     detector_model: Optional[str] = None,
308     +     max_gpu_temp_c: Optional[float] = None,
309     ) -> Dict[str, Any]:
310         """One golden video -> a feature record (shuttle + person features per candidate
window).
311
312     @@ -77,11 +174,18 @@ def extract_video(
313         # (parity with the shuttle net-crossing gate) rather than hard-coding an
orientation.
314         axis, net_px, _span = rally_gate.estimate_play_axis(points)
315         video_path = _derive_video_path(spec)
316     +     if not os.path.isfile(video_path):
317     +         raise FileNotFoundError(
318     +             f"{spec.get('name')}: source video not found for fusion extraction:
{video_path}"
319     +         )
320
321         if detector is None:
322             from backend.pipeline.detectors.person_detector import PersonDetector
323
324         -         detector = PersonDetector({"indexing": {"use_gpu": True}})
325     +         indexing_cfg: Dict[str, Any] = {"use_gpu": use_gpu}

```

```

326 +         if detector_model:
327 +             indexing_cfg["detector_model"] = detector_model
328 +             detector = PersonDetector({"indexing": indexing_cfg})
329
330 # Single-pass person detection over ALL candidate windows (open the video ONCE,
read
331 # forward ? no per-window seek). Then compute features per window (pure, no GPU).
332 @@ -95,8 +199,17 @@ def extract_video(
333     state = {"last": t0}
334
335     def _progress(done: int, total: int) -> None:
336 +         rec = None
337         if telemetry is not None:
338             telemetry.snapshot(done, total, phase=spec["name"])
339 +         rec = telemetry.snapshot(done, total, phase=spec["name"])
340 +         if max_gpu_temp_c is not None and rec is not None:
341 +             gpu = rec.get("gpu") or {}
342 +             temp = gpu.get("temp_c")
343 +             if temp is not None and float(temp) >= max_gpu_temp_c:
344 +                 raise RuntimeError(
345 +                     f"{spec['name']}: GPU temperature {float(temp):.1f}C reached "
346 +                     f"--max-gpu-temp-c {max_gpu_temp_c:.1f}; aborting before thermal
shutdown"
347 +                 )
348             now = time.monotonic()
349             if now - state["last"] >= log_every_sec or done >= total:
350                 el = now - t0
351 @@ -160,8 +273,23 @@ def run(
352     iou: float = 0.5,
353     fresh: bool = False,
354     smallest_first: bool = False,
355 +     source_manifest: Optional[str] = None,
356 +     only: Sequence[str] = (),
357 +     missing_fusion_only: bool = False,
358 +     use_gpu: bool = True,
359 +     detector_model: Optional[str] = None,
360 +     max_gpu_temp_c: Optional[float] = None,
361 ) -> List[str]:
362     specs = load_resolved_manifest(manifest_path)
363 +     source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
364 +     if source_manifest:
365 +         updated = attach_source_video_paths(specs, source_manifest)
366 +         print(
367 +             f"[fusion_golden] attached {updated}/{len(specs)} source video path(s) "
368 +             f"from {source_manifest}",
369 +             flush=True,
370 +         )
371 +     specs = _filter_specs(specs, only)
372     if smallest_first:
373         # Process small videos first for quick feedback (the slow full match lands
last);
374         # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
375 @@ -187,9 +315,19 @@ def run(
376     written: List[str] = []
377     for vi, spec in enumerate(specs, 1):
378         out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
379 -         # Resumable: skip a video whose JSON already exists (a re-run picks up where a
prior
380 -         # run left off instead of re-detecting). --fresh forces a redo.
381 -         if not fresh and os.path.isfile(out):
382 +         existing = os.path.isfile(out)
383 +         existing_full = existing and is_full_fusion_feature_file(out)
384 +         # Resumable: skip a video whose full-fusion JSON already exists. A leaner
385 +         # windowing-only JSON is stale for A7, so re-extract it even without --fresh.

```



```

386 +         if missing_fusion_only and not fresh and existing_full:
387 +             print(
388 +                 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: full-fusion "
389 +                 f"features already present -> {out}",
390 +                 flush=True,
391 +             )
392 +             written.append(out)
393 +             continue
394 +         if not fresh and existing and existing_full:
395 +             print(
396 +                 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already extracted
397 +                 f"-> {out} (use --fresh to redo)",
398 @@ -197,6 +335,12 @@ def run(
399 +             )
400 +             written.append(out)
401 +             continue
402 +         if not fresh and existing and not existing_full:
403 +             print(
404 +                 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: existing JSON "
405 +                 f"is not full-fusion schema; re-extracting...",
406 +                 flush=True,
407 +             )
408 +             print(
409 +                 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
410 +                 flush=True,
411 @@ -205,7 +349,15 @@ def run(
412 +             if telem is not None:
413 +                 telem.event("video_start", spec["name"])
414 +             try:
415 -                 data = extract_video(spec, frame_skip=frame_skip, iou=iou, telemetry=telem)
416 +                 data = extract_video(
417 +                     spec,
418 +                     frame_skip=frame_skip,
419 +                     iou=iou,
420 +                     telemetry=telem,
421 +                     use_gpu=use_gpu,
422 +                     detector_model=detector_model,
423 +                     max_gpu_temp_c=max_gpu_temp_c,
424 +                 )
425 +             except Exception as e: # noqa: BLE001 - record machine state at death, then
re-raise
426 +                 if telem is not None:
427 +                     telem.event("error", spec["name"], err=str(e))
428 @@ -246,6 +398,47 @@ def main() -> None:
429 +         action="store_true",
430 +         help="process smaller videos first for quick feedback (LOVO is order-
independent)",
431 +     )
432 +     ap.add_argument(
433 +         "--source-manifest",
434 +         default=None,
435 +         help=(
436 +             "collector golden_source_videos.json; overrides video paths for
source/proxy "
437 +             f"videos (or set {SOURCE_MANIFEST_ENV})"
438 +         ),
439 +     )
440 +     ap.add_argument(
441 +         "--only",
442 +         action="append",
443 +         default=[],
444 +         metavar="VIDEO_ID",
445 +         help="process one manifest video by name; repeatable",
446 +     )

```

```

447 +     ap.add_argument(
448 +         "--missing-fusion-only",
449 +         action="store_true",
450 +         help="skip videos whose existing JSON already has full shuttle+person fusion
schema",
451 +     )
452 +     ap.add_argument(
453 +         "--cpu",
454 +         action="store_true",
455 +         help="run torchvision person detection on CPU instead of CUDA",
456 +     )
457 +     ap.add_argument(
458 +         "--detector-model",
459 +         default=None,
460 +         choices=[
461 +             "fasterrcnn_resnet50_fpn",
462 +             "fasterrcnn_mobilenet_v3_large_fpn",
463 +             "retinanet_resnet50_fpn",
464 +         ],
465 +         help="torchvision detector model; default keeps the configured Faster R-CNN
ResNet",
466 +     )
467 +     ap.add_argument(
468 +         "--max-gpu-temp-c",
469 +         type=float,
470 +         default=None,
471 +         help="abort extraction if telemetry sees GPU temperature at or above this
value",
472 +     )
473 +     args = ap.parse_args()
474 +     run(
475 +         args.manifest,
476 @@ -254,6 +447,12 @@ def main() -> None:
477         args.iou,
478         fresh=args.fresh,
479         smallest_first=args.smallest_first,
480 +         source_manifest=args.source_manifest,
481 +         only=args.only,
482 +         missing_fusion_only=args.missing_fusion_only,
483 +         use_gpu=not args.cpu,
484 +         detector_model=args.detector_model,
485 +         max_gpu_temp_c=args.max_gpu_temp_c,
486     )
487
488
489 diff --git a/docs/ALPHA_LAUNCH_READINESS.md b/docs/ALPHA_LAUNCH_READINESS.md
490 index 5f550386..ea59c09b 100644
491 --- a/docs/ALPHA_LAUNCH_READINESS.md
492 +++ b/docs/ALPHA_LAUNCH_READINESS.md
493 @@ -1,6 +1,6 @@
494 # Alpha Launch Readiness Plan
495
496 -> **Status:** `DRAFT` - cross-repo execution tracker; **Owner:** @avidullu;
**Created:** 2026-07-03; **Last updated:** 2026-07-03
497 +> **Status:** `DRAFT` - cross-repo execution tracker; **Owner:** @avidullu;
**Created:** 2026-07-03; **Last updated:** 2026-07-04
498     > **Lifecycle:** `DRAFT` -> IN PROGRESS -> DONE -> archived`
499     > **Tracking anchors:** Section 7 is the progress tracker and source of truth; indexed
in `docs/README.md` and `docs/DOC_STATUS.md`.
500     > **Relation to existing docs:** consolidates the launch-relevant pending items from
`NEXT_STEPS.md`, `ROADMAP.md`, `docs/data_pipeline/GOLDEN_VIDEOS.md`,
`R2_EVAL_METRIC_DESIGN.md`, `COMPUTE_DECOUPLED_SERVING/`, and the sibling `kheIsutra` Studio
docs.
501     @@ -10,7 +10,7 @@
502

```

503 ## 0. TL;DR
504
505 -The corpus is now large enough to move from "grow labels first" into alpha-readiness work, but the 15-video headline is not itself a launch gate pass. `[verified]` The golden manifest has 15 badminton videos; `[analysis]` the local manifest needed path rebasing before manifest-driven commands ran; `[analysis]` only 6 of 15 feature JSONs had the full fusion schema; `[analysis]` the n=15 quality baseline is stale and must be intentionally refreshed before it can gate anything.
506 +The corpus is now large enough to move from "grow labels first" into alpha-readiness work, but the 15-video headline is not itself a launch gate pass. `[verified]` The golden manifest has 15 badminton videos; `[analysis]` A2-A7 made the local manifest/path and full-fusion/audio feature surface reproducible on all 15; `[analysis]` A4/A5 refreshed the n=15 nightly and heuristic floors; `[analysis]` A8/A9 still reject served Gate A default promotion and owned-model serving claims on the current evidence.
507
508 The first controlled alpha should be a hosted, authenticated product loop: upload a video, process it, select rallies, compile, and download a reel. The owned-model/commercial rally-detection claim remains a separate NO-GO until the ship-gate target is ratified and the WASB checkpoint provenance is resolved.
509
510 @@ -31,8 +31,9 @@ This doc separates those into executable workstreams. The goal is to reach an in
511 | D1 | **Product alpha and owned-model launch are separate decisions.** |
`NEXT\_STEPS.md` NO-GO banner, 2026-06-30; verified 2026-07-03 | We can launch a controlled hosted product loop without claiming the owned model is commercially cleared. |
512 | D2 | **First alpha must be page-driven, not SSH/CLI-driven.** | `NEXT\_STEPS.md` P0 UI-driven/shareable item; verified 2026-07-03 | Hosted service, edge auth, upload, job polling, compile, and download are launch-critical. |
513 | D3 | **The 15-video corpus is now a measurement asset, not a free launch pass.** | `[verified]` `docs/data\_pipeline/GOLDEN\_VIDEOS.md`; `[analysis]` local eval runs, 2026-07-03 | Rebase/portability, feature completeness, baseline refresh, and gate ratification come before quality claims. |
514 -| D4 | **Do not promote served Gate A by default from the current n=15 evidence.** | `[analysis]` `backend.eval.serve\_contrast` run, 2026-07-03 | Proposal-side lift does not survive served-windowing safety; keep it as an opt-in/analysis lever until owner ratification. |
515 +| D4 | **Do not promote served Gate A by default from the current n=15 evidence.** | `[analysis]` `backend.eval.serve\_contrast` run, 2026-07-04 | Proposal-side lift does not survive served-windowing safety; keep it as an opt-in/analysis lever until owner ratification. |
516 | D5 | **P10/P11 corpus promotion and train/re-eval are alpha-plus unless owner reopens D13.** | sibling `khealsutra/docs/STUDIO\_MEDIA\_LIFECYCLE.md`; verified 2026-07-03 | First alpha can use manual/offline corpus promotion; automated flywheel follows after serving and gates are stable. |
517 +| D6 | **Gen-0 n=15 shadow training is non-serving.** | `[analysis]` `training.gen0.harness` run, 2026-07-04 | Learned mean beats the heuristic floor, but the paired sign test is not significant; #172 and WASB C3 remain hard blockers. |
518
519 ## 3. Verified snapshot, 2026-07-03
520
521 @@ -50,10 +51,11 @@ Corpus/eval facts:
522
523 - `[verified]` `docs/data\_pipeline/GOLDEN\_VIDEOS.md` says
`output/human\_lovo\_manifest.json` is the ingested golden corpus at `n = 15`.
524 - `[analysis]` The local manifest had stale absolute paths. A temporary rebased manifest found all 15 trajectories and labels in this workspace.
525 -- `[analysis]` There are 15 `output/\*\_golden\_features.json` files, but only 6 carry full shuttle/person/audio candidate features. The newer 9 are windowing-only schema and are skipped by fusion/person eval.
526 -- `[analysis]` `python -m backend.eval.fusion\_compare --features-dir output` loaded only 6 videos and measured person-fusion `Delta F1 = -0.021`; not promotable.
527 -- `[analysis]` `python -m backend.eval.serve\_contrast --manifest <rebased-n15>` measured proposal Gate A mean `Delta F1 +0.0287`, but served mean `Delta F1 -0.0328`; not served-safe.
528 -- `[analysis]` `python -m training.gen0.harness --manifest <rebased-n15> --floor eval\_baselines/heuristic\_lovo\_n6.json --version analysis-n15 --out <temp-out>` produced learned LOVO mean F1 `0.551`, `ship=False`; this pre-A5 run used the stale n=6 floor, so future

comparisons should use `eval\_baselines/heuristic\_lovo\_n15.json`.

529 +- `[analysis]` A7 generated full shuttle/person fusion schema for all 15 videos and A8 audio augmentation populated audio features for all 15.

530 +- `[analysis]` `python -m backend.eval.fusion\_compare --features-dir output` now runs over all 15 and measures person-fusion `Delta F1 = +0.008`; the paired CI crosses zero, so person fusion is not promotable by itself.

531 +- `[analysis]` `python -m backend.eval.fusion\_audio --features-dir output --compare` measures audio `Delta F1 = +0.018`; the paired CI crosses zero, so audio is a shadow/suggestive cue, not a default-flip.

532 +- `[analysis]` `python -m backend.eval.serve\_contrast --manifest output\human\_lovo\_manifest.json` measured proposal Gate A mean `Delta F1 +0.0287`, but served mean `Delta F1 -0.0328`; not served-safe.

533 +- `[analysis]` `python -m training.gen0.harness --manifest output\human\_lovo\_manifest.json --floor eval\_baselines\heuristic\_lovo\_n15.json --version 0.1.1-n15-shadow.20260704 --out artifacts\rally\_tal\_gen0` produced learned LOVO mean F1 `0.551`, floor passed versus heuristic `0.446`, but sign test `9/15`, `p = 0.3036`, `ship=False`.

534 - `[analysis]` `python scripts/nightly\_regression.py --manifest <rebased-n15> --features-dir output --no-report` reported default/milestone tolF1 `0.3636` vs baseline-off tolF1 `0.1382`, but marked the frozen baseline stale due to config/corpus changes.

535 - `[analysis]` Before A6, `python -m backend.eval.ablation --features-dir output --granularity group` failed at import due to a circular import between `backend/eval/ablation.py` and `backend/eval/ablation\_pdf.py`; A6 fixes the module entrypoint and keeps full-fusion ablation scoped to the 6 videos with full schema until A7.

536

537 @@ -97,13 +99,13 @@ Legend: TODO = not started, IN PROGRESS = actively being worked, DONE = shipped/

538 | A0 | Create this alpha readiness tracker and index it in docs | `badminton-highlight-indexer` | - | No | DONE | #451 |

539 | A1 | Stand up alpha serving endpoint: Cloud Run/GPU or approved host, health check, edge auth ON, upload -> process -> jobs -> compile -> download verified | `khealsutra` + `badminton-highlight-indexer` | A0 | Owner spend / CF config | TODO | - |

540 | A2 | Make the n=15 golden manifest portable/reproducible; add a path/corpus smoke check that fails loudly when labels or trajectories are missing | `badminton-highlight-indexer` | A0 | No | DONE | #452 |

541 -| A3 | Promote the 15-video source-video/MD5 records into the collector/vault path; reconcile collector's six-video source manifest with the 15-video eval corpus | `sports-data-collector` + vault | A2 | Owner upload/storage scope | TODO | - |

542 +| A3 | Promote the 15-video source-video/MD5 records into the collector/vault path; reconcile collector's six-video source manifest with the 15-video eval corpus | `sports-data-collector` + vault | A2 | Owner upload/storage scope | IN PROGRESS | collector #62 |

543 | A4 | Refresh the n=15 nightly regression baseline intentionally and record the exact command/output; keep stale-baseline warnings until reviewed | `badminton-highlight-indexer` | A2 | Reviewer sign-off | DONE | #453 |

544 | A5 | Recompute the heuristic n=15 floor and replace the stale `heuristic\_lovo\_n6` floor for future Gen-0 comparisons | `badminton-highlight-indexer` | A2 | No | DONE | #454 |

545 -| A6 | Fix `backend.eval.ablation` CLI circular import, then run the R2/tolF1 ablation on the n=15 corpus | `badminton-highlight-indexer` | A2 | No | IN PROGRESS | this PR |

546 -| A7 | Regenerate or upconvert full-fusion feature JSONs for the newer 9 videos so fusion/person/audio analyses run over all 15 | `badminton-highlight-indexer` | A2 | GPU/data availability | TODO | - |

547 -| A8 | Re-run `fusion\_compare`, group ablation, and served contrast after A7; explicitly promote/reject person fusion, Gate A served-default, and any other R-series default flips | `badminton-highlight-indexer` | A6, A7 | Reviewer sign-off | TODO | - |

548 -| A9 | Run Gen-0/TemporalMaxer shadow training against the refreshed n=15 floor; produce a non-serving artifact bundle and gate verdict | `badminton-highlight-indexer` | A5, A8 | Owner approval for M0.4 scope | TODO | - |

549 +| A6 | Fix `backend.eval.ablation` CLI circular import, then run the R2/tolF1 ablation on the n=15 corpus | `badminton-highlight-indexer` | A2 | No | DONE | #455 |

550 +| A7 | Regenerate or upconvert full-fusion feature JSONs for the newer 9 videos so fusion/person/audio analyses run over all 15 | `badminton-highlight-indexer` | A2 | GPU/data availability | IN PROGRESS | this PR |

551 +| A8 | Re-run `fusion\_compare`, group ablation, and served contrast after A7; explicitly promote/reject person fusion, Gate A served-default, and any other R-series default flips | `badminton-highlight-indexer` | A6, A7 | Reviewer sign-off | IN PROGRESS | this PR |

552 +| A9 | Run Gen-0/TemporalMaxer shadow training against the refreshed n=15 floor;

```

produce a non-serving artifact bundle and gate verdict | `badminton-highlight-indexer` | A5, A8
| Owner approval for M0.4 scope | IN PROGRESS | this PR |
553 | A10 | Decide and record the alpha ship-gate target: metric, corpus slice, minimum
threshold, significance rule, and what claims are allowed | Owner + `badminton-highlight-
indexer` | A4, A5, A8 | Owner decision | GATED | - |
554 | A11 | Resolve or explicitly defer WASB C3 provenance for alpha messaging; if
deferred, document "no commercial model sharing" copy guardrail | Owner/legal | A10 |
Legal/provenance | GATED | - |
555 | A12 | Reopen or defer P10/P11 corpus flywheel: `PATCH /api/segments`, collector
promotion, `train`/`reeval` job, delta-F1 surface, and alignment with
`COMPUTE_DECOUPLED_SERVING` M11 | `kheksutra` + `badminton-highlight-indexer` + collector | A1,
A3 | Owner D13 reopen (`kheksutra/docs/STUDIO_MEDIA_LIFECYCLE.md`) | GATED | - |
556 @@ -174,22 +176,57 @@ python -m backend.eval.rally_seg_eval --manifest
output\human_lovo_manifest.json
557
558 Result: the ablation CLI now runs, but full-fusion feature ablation still reports `n=6`
because only the original six feature JSONs carry the full fusion schema; A7 must
regenerate/upconvert the newer nine before fusion/person/audio promotion decisions. The n=15
R2/tolF1 contrast is positive for the served milestone versus baseline-off: baseline/off `tolF1
= 0.138`, served `tolF1 = 0.364`, `Delta tolF1 = +0.225` with CI `[+0.165, +0.283]`, sign
`14/0`, `p = 0.000`; `Delta F1@0.5 = +0.236` with CI `[+0.161, +0.313]`; merge rate improves
from `0.638` to `0.135`. Net over-segmentation guard passes (`1.276 -> 0.378`), while the strict
segment-count rule remains blocked by increased splits, so R2/tolF1 is the supported gate for
the baseline-off to served milestone contrast rather than a blanket fusion/default-flip
approval.
559
560 +A7 full-fusion/audio feature refresh record:
561 +
562 +```powershell
563 +python -m backend.eval.fusion_golden --manifest output\human_lovo_manifest.json --out-
dir output --source-manifest ..\sports-data-collector\data\golden_source_videos.json --missing-
fusion-only --smallest-first --max-gpu-temp-c 87
564 +python -m backend.eval.fusion_audio --manifest output\human_lovo_manifest.json
--features-dir output --source-manifest ..\sports-data-collector\data\golden_source_videos.json
--augment
565 +```
566 +
567 +Result: the GPU pass completed without a thermal abort after fans were set to max
performance. Full-fusion schema is now present for `15/15` feature JSONs, and audio features are
present for `15/15`. The large regenerated clips completed as follows: `mahadevpura_1` `1`
window / `0` positives / `10` GT, `GX020094` `99` / `30` / `40`, `GX030094` `79` / `21` / `41`,
`GX010141` `26` / `11` / `29`, `GX010137` `56` / `31` / `34`, `Boxhill_Doubles` `59` / `12` /
`43`, `mahadevpura_2` `14` / `2` / `40`, `GX010128` `51` / `15` / `52`, `Badminton_BXH_2` `29` /
`1` / `44`. The source manifest supplied all 15 source-video paths, including the F-drive
Boxhill originals.
568 +
569 +A8 fusion/default-decision record:
570 +
571 +```powershell
572 +python -m backend.eval.fusion_compare --features-dir output
573 +python -m backend.eval.fusion_audio --features-dir output --compare
574 +python -m backend.eval.ablation --features-dir output --granularity group
575 +python -m backend.eval.serve_contrast --manifest output\human_lovo_manifest.json
576 +python -m backend.eval.rally_seg_eval --manifest output\human_lovo_manifest.json
--features-dir output --preset served --max-window-duration 0 --inactivity-end 0 --vs served
--vs-max-window-duration 6 --vs-inactivity-end 1.5 --json-out output\A6_r2_ablation.json
577 +```
578 +
579 +Result:
580 +
581 +- Person fusion is **not promoted**: shuttle-only F1 `0.302`, shuttle+person F1
`0.311`, `Delta F1 +0.008`, paired CI `[-0.040, +0.055]`, sign `8W/6L`, `p = 0.791`.
582 +- Audio is **shadow/suggestive, not promoted**: shuttle+person F1 `0.311`,
shuttle+person+audio F1 `0.329`, `Delta F1 +0.018`, paired CI `[-0.017, +0.053]`, sign `10W/4L`,
`p = 0.180`.

```

```

583     +- Group ablation remains non-significant for cue promotion: audio `+0.018` CI `[-0.017,
+0.053]`; shuttle `+0.013` CI `[-0.002, +0.029]`; gate_a `-0.001` CI `[-0.012, +0.008]`; person
`-0.009` CI `[-0.045, +0.021]`.
584     +- Gate A served default is **rejected**: proposal mean `Delta F1 +0.0287`, CI
`[-0.0222, +0.0680]`; served mean `Delta F1 -0.0328`, CI `[-0.0799, +0.0052]`, `regresses=True`.
585     +- R2/cap/inactivity remains the strong quality result: served candidate F1 `0.472` vs
baseline-off `0.237`, `Delta F1 +0.236`, CI `[+0.161, +0.313]`; tolF1 `0.364` vs `0.138`, `Delta
tolF1 +0.225`, CI `[+0.165, +0.283]`; merge rate `0.135` vs `0.638`. Net over-seg guard passes,
strict per-video split-count guard still blocks blanket promotion language.
586     +
587     +A9 Gen-0 shadow-training record:
588     +
589     +```powershell
590     +python -m training.gen0.harness --manifest output\human_lovo_manifest.json --floor
eval_baselines\heuristic_lovo_n15.json --out artifacts\rally_tal_gen0 --version
0.1.1-n15-shadow.20260704
591     +```
592     +
593     +Result: current Gen-0 LogReg/TAL harness produced local bundle
`artifacts\rally_tal_gen0\0.1.1-n15-shadow.20260704` with learned LOVO mean F1 `0.551` versus
heuristic floor `0.4456`, so `floor_passed=True`. The gate remains **non-serving**: paired sign
test `9/15`, `p = 0.3036`, `ship=False`; blockers are uncalibrated ship-gate target #172 and
WASB checkpoint provenance C3. TemporalMaxer itself is not implemented in this branch; the
evidence supports either closing A9 as a Gen-0 shadow baseline or opening a follow-up M0.4
scorer-swap PR.
594     +
595     ## 9. Open questions
596
597     1. **Owner:** Is the first alpha allowed to use manual/offline corpus promotion, or
should P10/P11 be reopened before any tester wave?
598     2. **Owner/platform:** Which hosted compute target is the next stand-up: Cloud Run GPU,
GCE L4 VM, or another approved GPU host?
599     3. **Owner/legal:** What is the acceptable alpha wording while WASB C3 remains
unresolved?
600     4. **Engineering:** Should the portable corpus manifest live as a generated artifact, a
checked-in redacted manifest, or both?
601     -5. **Engineering:** Should n=15 feature regeneration target full fusion schema for
every clip, or should the eval loaders support mixed schemas with explicit capability tags?
602     +5. **Engineering:** Should A9 close on the current Gen-0 non-serving bundle, or should
the actual TemporalMaxer scorer swap be implemented before marking A9 done?
603
604     ## 10. Definition of done
605
606     - [ ] Invited tester can complete the hosted record/upload -> reel download CUJ without
SSH.
607     - [ ] Alpha service is gated behind the chosen auth posture and has a documented
disable/rollback path.
608     -- [ ] n=15 corpus is portable enough that manifest-driven quality commands do not rely
on stale local absolute paths.
609     -- [ ] n=15 nightly baseline and heuristic floor are refreshed with recorded commands.
610     -- [ ] R2/tolF1 gate is promoted or rejected with a recorded ablation result.
611     -- [ ] Fusion/person/Gate-A served-default decisions are explicitly promoted or rejected
on the 15-video corpus.
612     +- [x] n=15 corpus is portable enough that manifest-driven quality commands do not rely
on stale local absolute paths.
613     +- [x] n=15 nightly baseline and heuristic floor are refreshed with recorded commands.
614     +- [x] R2/tolF1 gate is promoted or rejected with a recorded ablation result.
615     +- [x] Fusion/person/Gate-A served-default decisions are explicitly promoted or rejected
on the 15-video corpus.
616     - [ ] Ship-gate target and allowed alpha claims are recorded.
617     - [ ] WASB C3 status is recorded and reflected in alpha messaging.
618     - [ ] P10/P11 are either reopened with a committed sequence or explicitly deferred for
alpha-plus.
619     @@ -226,3 +263,4 @@ python -m backend.eval.rally_seg_eval --manifest
output\human_lovo_manifest.json

```

```

620     ### Changelog
621
622     - 2026-07-03 - Created from a synced five-repo review of `NEXT_STEPS`, `ROADMAP`,
golden-corpus docs, local eval artifacts, and the current 15-video corpus state.
623     +- 2026-07-04 - Recorded A7 full-fusion/audio regeneration, A8 n15 fusion/default
decisions, and A9 Gen-0 n15 shadow-training gate verdict.
624     diff --git a/tests/test_ablation.py b/tests/test_ablation.py
625     index a4491d7c..b5450f7b 100644
626     --- a/tests/test_ablation.py
627     +++ b/tests/test_ablation.py
628     @@ -10,9 +10,10 @@
629     - the honest-stats contract ? every row ships a CI + sign test, and no verdict is
stronger
630         than the stats support (the no-over-claim contract).
631     - a valid non-trivial PDF is produced (asserted via pypdf page count > 0).
632     - the golden litmus reproduction ? on the 6 real ``output/*_golden_features.json``
the runner
633     - reproduces the hand-measured Gate-A result (?F1 ? +0.074, 6/6 wins, p ? 0.031).
Skips
634     - cleanly when the golden JSONs are absent (CI), runs when present (owner box).
635     + - the golden litmus reproduction ? on real ``output/*_golden_features.json`` files
the runner
636     + reproduces the hand-measured Gate-A result for either the legacy n=6 substrate or
the refreshed
637     + n=15 A7 substrate. Skips cleanly when the golden JSONs are absent (CI), runs when
present
638     + (owner box).
639     ""
640
641     from __future__ import annotations
642     @@ -517,7 +518,7 @@ def test_load_videos_skips_windowing_only_schema(tmp_path):
643
644
645     def _golden_present() -> bool:
646     -     ""True iff ?6 FULL-FUSION-SCHEMA golden files exist (the litmus needs the
shuttle/person cue
647     +     ""True iff >=6 FULL-FUSION-SCHEMA golden files exist (the litmus needs the
shuttle/person cue
648         columns). Counts only ablatable files ? leaner velocity-windowing files are skipped
by
649         ``load_videos``, so counting them would trip the litmus on an un-ablatable corpus
(the n=11
650         mixed-schema bug).""
651     @@ -540,29 +541,42 @@ def _golden_present() -> bool:
652         reason="golden output/*_golden_features.json absent (run fusion_golden first)",
653     )
654     def test_litmus_reproduces_gate_a():
655     -     ""LITMUS: on the 6 real golden feature JSONs the runner reproduces the hand-
measured Gate-A
656     -     result ? marginal ?F1 ? +0.074, 6/6 wins, p ? 0.031, CI ? [+0.042, +0.104], within
tolerance.
657     -     This validates the whole framework against the recorded ground truth. Skips cleanly
in CI
658     -     where the golden substrate is absent; runs on the owner box where it is present.""
659     +     ""LITMUS: on real golden feature JSONs the runner reproduces the recorded Gate-A
result.
660     +
661     +     The legacy n=6 owner-box substrate preserves the 2026-06-14 hand measurement. After
A7, the
662     +     refreshed owner-box substrate has n=15 full-fusion files; that result is
intentionally weaker
663     +     and supports "keep structural, do not promote served default".
664     +     ""
665     videos = ablation.load_videos(GOLDEN_DIR)
666     -     assert len(videos) == 6

```

```

667 +     assert len(videos) in {6, 15}
668         cfg = ablation.litmus_config()
669         report = run_ablation(videos, cfg)
670         assert len(report.rows) == 1
671         row = report.rows[0]
672         assert row.signal == "gate_a"
673 -     # The recorded QUALITY_ITERATIONS truth (2026-06-14), reproduced within tolerance.
674 -     assert row.delta_f1 == pytest.approx(0.074, abs=0.005)
675 -     assert row.sign_wins == 6 and row.sign_losses == 0
676 -     assert row.sign_p_value == pytest.approx(0.031, abs=0.005)
677 -     assert row.ci_low == pytest.approx(0.042, abs=0.01)
678 -     assert row.ci_high == pytest.approx(0.104, abs=0.01)
679 -     assert row.ci_excludes_zero is True
680 +     if len(videos) == 6:
681 +         # The recorded QUALITY_ITERATIONS truth (2026-06-14), reproduced within
tolerance.
682 +         assert row.delta_f1 == pytest.approx(0.074, abs=0.005)
683 +         assert row.sign_wins == 6 and row.sign_losses == 0
684 +         assert row.sign_p_value == pytest.approx(0.031, abs=0.005)
685 +         assert row.ci_low == pytest.approx(0.042, abs=0.01)
686 +         assert row.ci_high == pytest.approx(0.104, abs=0.01)
687 +         assert row.ci_excludes_zero is True
688 +         # C4: over-segmentation drops 1.68 -> 1.01 when Gate-A is on (full).
689 +         assert row.seg_ratio_full == pytest.approx(1.01, abs=0.05)
690 +         assert row.seg_ratio_ablated == pytest.approx(1.68, abs=0.05)
691 +     else:
692 +         # The A7 refreshed n=15 substrate: proposal-side structural signal, but CI
crosses zero.
693 +         assert row.delta_f1 == pytest.approx(0.0287, abs=0.005)
694 +         assert row.sign_wins == 12 and row.sign_losses == 1
695 +         assert row.sign_p_value == pytest.approx(0.0034, abs=0.002)
696 +         assert row.ci_low == pytest.approx(-0.020, abs=0.01)
697 +         assert row.ci_high == pytest.approx(0.066, abs=0.01)
698 +         assert row.ci_excludes_zero is False
699 +         assert row.seg_ratio_full == pytest.approx(0.79, abs=0.05)
700 +         assert row.seg_ratio_ablated == pytest.approx(1.36, abs=0.05)
701         # Gate-A is structural ? reported but never auto-promoted to earns_keep.
702         assert row.verdict == ablation.STRUCTURAL_PROTECTED
703 -     # C4: over-segmentation drops 1.68 ? 1.01 when Gate-A is on (full).
704 -     assert row.seg_ratio_full == pytest.approx(1.01, abs=0.05)
705 -     assert row.seg_ratio_ablated == pytest.approx(1.68, abs=0.05)
706
707
708     @pytest.mark.skipif(
709     @@ -572,7 +586,7 @@ def test_litmus_report_is_well_formed():
710         """The litmus report is a proper AblationReport (provenance + a sorted single
row)."""
711         report = run_ablation(ablation.load_videos(GOLDEN_DIR), ablation.litmus_config())
712         assert isinstance(report, AblationReport)
713 -     assert report.num_videos == 6
714 +     assert report.num_videos in {6, 15}
715         assert report.full_set_signals == ["gate_a"]
716         assert report.label_set_hash
717         out = ablation.format_table(report)
718         diff --git a/tests/test_audio_features.py b/tests/test_audio_features.py
719         index 26028591..d962b5e4 100644
720         --- a/tests/test_audio_features.py
721         +++ b/tests/test_audio_features.py
722         @@ -118,6 +118,7 @@ def
test_augment_rebases_stale_manifest_before_deriving_video_path(tmp_path, mon
723             out.mkdir(parents=True)
724             traj.write_text("x\n", encoding="utf-8")
725             labels.write_text("start_time,end_time\n0,1\n", encoding="utf-8")
726 +         labels.with_name("Foo.MP4").write_text("video\n", encoding="utf-8")
727             manifest = out / "human_lovo_manifest.json"

```



```

728     manifest.write_text(
729         json.dumps(
730 @@ -163,6 +164,80 @@ def _fake_detect(video_path, k=2.5):
731     assert data["candidates"][0]["audio"]["audio_hit_count"] == 1.0
732
733
734 +def test_augment_uses_source_manifest_video_path(tmp_path, monkeypatch):
735 +    workspace = tmp_path / "workspace"
736 +    repo = workspace / "badminton-highlight-indexer"
737 +    out = repo / "output"
738 +    traj = workspace / "Annotation Setup" / "Trajectories" / "foo.csv"
739 +    labels = workspace / "Annotation Setup" / "Golden Labelled" / "Foo.rallies.csv"
740 +    source_video = workspace / "source" / "Foo_proxy.mp4"
741 +    traj.parent.mkdir(parents=True)
742 +    labels.parent.mkdir(parents=True)
743 +    source_video.parent.mkdir(parents=True)
744 +    out.mkdir(parents=True)
745 +    traj.write_text("x\n", encoding="utf-8")
746 +    labels.write_text("start_time,end_time\n0,1\n", encoding="utf-8")
747 +    source_video.write_text("video\n", encoding="utf-8")
748 +    manifest = out / "human_lovo_manifest.json"
749 +    manifest.write_text(
750 +        json.dumps(
751 +            [
752 +                {
753 +                    "name": "foo",
754 +                    "trajectory": str(traj),
755 +                    "labels": str(labels),
756 +                    "fps": 30.0,
757 +                    "frame_width": 1920.0,
758 +                }
759 +            ],
760 +        ),
761 +        encoding="utf-8",
762 +    )
763 +    source_manifest = workspace / "golden_source_videos.json"
764 +    source_manifest.write_text(
765 +        json.dumps(
766 +            {
767 +                "schema": "sports-data-collector.golden_source_videos.v2",
768 +                "videos": [
769 +                    {
770 +                        "video_id": "foo",
771 +                        "local_path": str(source_video),
772 +                        "source_status": "present_local",
773 +                    }
774 +                ],
775 +            },
776 +        ),
777 +        encoding="utf-8",
778 +    )
779 +    feature_json = out / "foo_golden_features.json"
780 +    feature_json.write_text(
781 +        json.dumps(
782 +            {
783 +                "name": "foo",
784 +                "fps": 30.0,
785 +                "frame_width": 1920.0,
786 +                "candidates": [{"start": 0.0, "end": 1.0}],
787 +                "gts": [[0.0, 1.0]],
788 +            },
789 +        ),
790 +        encoding="utf-8",
791 +    )
792 +    seen = []

```

```

793 +
794 +     def _fake_detect(video_path, k=2.5):
795 +         seen.append((video_path, k))
796 +         return [H(0.5, 3.0)]
797 +
798 +     monkeypatch.setattr(fusion_audio, "detect_hits_in_video", _fake_detect)
799 +
800 +     written = fusion_audio.augment(
801 +         str(manifest), str(out), k=1.5, source_manifest=str(source_manifest)
802 +     )
803 +
804 +     assert written == [str(feature_json)]
805 +     assert seen == [(str(source_video), 1.5)]
806 +
807 +
808     def test_audio_increment_compare(tmp_path):
809         d = str(tmp_path)
810         for k in range(6):
811             diff --git a/tests/test_fusion_compare.py b/tests/test_fusion_compare.py
812             index f042eea1..6b4226e6 100644
813             --- a/tests/test_fusion_compare.py
814             +++ b/tests/test_fusion_compare.py
815             @@ -181,6 +181,7 @@ def detections_over_windows(self, video, windows, frame_skip,
progress_cb=None):
816                 "fps": 30.0,
817                 "frame_width": 1920.0,
818                 "labels": "foo.rallies.csv",
819             +         "video_path": __file__,
820             }
821             data = fusion_golden.extract_video(
822                 spec, detector=_StubDetector(), frame_skip=3, iou=0.5
823             @@ -197,6 +198,47 @@ def detections_over_windows(self, video, windows, frame_skip,
progress_cb=None):
824                 assert all(isinstance(v, float) for v in c0["person"].values())
825
826
827 +def test_extract_video_aborts_on_max_gpu_temp(monkeypatch):
828 +    from backend.eval import fusion_golden
829 +    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
830 +
831 +    monkeypatch.setattr(
832 +        fusion_golden.distill_local,
833 +        "build_candidates",
834 +        lambda traj, fps, fw: ([([0.0, 1.0]), [[1.0, 0.1, 0.1, 0.1, 0.0]]]),
835 +    )
836 +    monkeypatch.setattr(
837 +        TrackNetRunner, "parse_trajectory_csv", staticmethod(lambda traj: [])
838 +    )
839 +    monkeypatch.setattr(fusion_golden, "load_golden_gts", lambda labels: [])
840 +
841 +    class _HotTelemetry:
842 +        def snapshot(self, done, total, phase=None):
843 +            return {"gpu": {"temp_c": 90.0}}
844 +
845 +    class _StubDetector:
846 +        def detections_over_windows(self, video, windows, frame_skip,
progress_cb=None):
847 +            progress_cb(1, 10)
848 +            return []
849 +
850 +    spec = {
851 +        "name": "hot",
852 +        "trajectory": "bar.csv",
853 +        "fps": 30.0,
854 +        "frame_width": 1920.0,

```

```

855 +         "labels": "foo.rallies.csv",
856 +         "video_path": __file__,
857 +     }
858 +
859 +     with pytest.raises(RuntimeError, match="GPU temperature 90.0C"):
860 +         fusion_golden.extract_video(
861 +             spec,
862 +             detector=_StubDetector(),
863 +             telemetry=_HotTelemetry(),
864 +             max_gpu_temp_c=85.0,
865 +         )
866 +
867 +
868 +     def test_fusion_golden_run_is_resumable(tmp_path, monkeypatch):
869 +         """run() skips a video whose JSON already exists (resume a long run); --fresh
870 +         redoes all."""
871 +         from backend.eval import fusion_golden
872 +         @@ -261,6 +303,121 @@ def _stub(spec, **kw):
873 +             assert extracted == ["alpha", "beta"] # --fresh redoes both
874 +
875 +     +def test_fusion_golden_run_reextracts_windowing_only_json(tmp_path, monkeypatch):
876 +         + """A7: an existing lean windowing JSON is stale for fusion and must be
877 +         regenerated."""
878 +         + from backend.eval import fusion_golden
879 +         +
880 +         + out_dir = str(tmp_path / "out")
881 +         + os.makedirs(out_dir)
882 +         + (tmp_path / "a.csv").write_text("x\n", encoding="utf-8")
883 +         + (tmp_path / "a.rallies.csv").write_text("x\n", encoding="utf-8")
884 +         + manifest = [
885 +         +     {
886 +         +         "name": "alpha",
887 +         +         "trajectory": "a.csv",
888 +         +         "fps": 30.0,
889 +         +         "frame_width": 1920.0,
890 +         +         "labels": "a.rallies.csv",
891 +         +     }
892 +         + ]
893 +         + manifest_path = str(tmp_path / "m.json")
894 +         + with open(manifest_path, "w", encoding="utf-8") as f:
895 +         +     json.dump(manifest, f)
896 +         + _write_windowing_video(out_dir, "alpha")
897 +         +
898 +         + extracted = []
899 +         +
900 +         + def _stub(spec, **kw):
901 +         +     extracted.append(spec["name"])
902 +         +     return {
903 +         +         "name": spec["name"],
904 +         +         "fps": 30.0,
905 +         +         "frame_width": 1920.0,
906 +         +         "candidates": [],
907 +         +         "gts": [],
908 +         +     }
909 +         +
910 +         + monkeypatch.setattr(fusion_golden, "extract_video", _stub)
911 +         +
912 +         + fusion_golden.run(manifest_path, out_dir, fresh=False)
913 +         +
914 +         + assert extracted == ["alpha"]
915 +         + assert fusion_golden.is_full_fusion_feature_file(
916 +         +     os.path.join(out_dir, "alpha_golden_features.json")
917 +         + )

```

```

918 +
919 +def test_fusion_golden_source_manifest_supplies_video_paths(tmp_path, monkeypatch):
920 +    """The collector source manifest can provide Drive/F-drive video paths for A7
921 +    extraction."""
922 +
923 +    from backend.eval import fusion_golden
924 +
925 +    out_dir = str(tmp_path / "out")
926 +    os.makedirs(out_dir)
927 +    for name in ("a.csv", "b.csv", "a.rallies.csv", "b.rallies.csv"):
928 +        (tmp_path / name).write_text("x\n", encoding="utf-8")
929 +    source_video = tmp_path / "source-alpha.mp4"
930 +    source_video.write_text("video\n", encoding="utf-8")
931 +    source_manifest = tmp_path / "golden_source_videos.json"
932 +    source_manifest.write_text(
933 +        json.dumps(
934 +            {
935 +                "schema": "sports-data-collector.golden_source_videos.v2",
936 +                "videos": [
937 +                    {
938 +                        "video_id": "alpha",
939 +                        "local_path": str(source_video),
940 +                        "source_status": "present_local",
941 +                    },
942 +                    {
943 +                        "video_id": "beta",
944 +                        "local_path": str(tmp_path / "missing.mp4"),
945 +                        "source_status": "missing_local_source_hash_from_run_log",
946 +                    },
947 +                ],
948 +            },
949 +            encoding="utf-8",
950 +        )
951 +    )
952 +    manifest = [
953 +        {
954 +            "name": "alpha",
955 +            "trajectory": "a.csv",
956 +            "fps": 30.0,
957 +            "frame_width": 1920.0,
958 +            "labels": "a.rallies.csv",
959 +            "video_path": str(tmp_path / "stale-alpha.mp4"),
960 +        },
961 +        {
962 +            "name": "beta",
963 +            "trajectory": "b.csv",
964 +            "fps": 30.0,
965 +            "frame_width": 1920.0,
966 +            "labels": "b.rallies.csv",
967 +        },
968 +    ]
969 +    manifest_path = str(tmp_path / "m.json")
970 +    with open(manifest_path, "w", encoding="utf-8") as f:
971 +        json.dump(manifest, f)
972 +
973 +    seen = []
974 +
975 +    def _stub(spec, **kw):
976 +        seen.append((spec["name"], spec.get("video_path")))
977 +        return {
978 +            "name": spec["name"],
979 +            "fps": 30.0,
980 +            "frame_width": 1920.0,
981 +            "candidates": [],
982 +            "gts": [],
983 +        }

```

```

982 +
983 + monkeypatch.setattr(fusion_golden, "extract_video", _stub)
984 +
985 + fusion_golden.run(manifest_path, out_dir, source_manifest=str(source_manifest))
986 +
987 + assert seen == [("alpha", str(source_video)), ("beta", None)]
988 +
989 +
990 def test_fusion_golden_smallest_first(tmp_path, monkeypatch):
991     """--smallest-first orders by trajectory size; default keeps manifest order."""
992     from backend.eval import fusion_golden
993

```

6. user (2026-07-04T00:44:51.402Z)

```

1     """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3     Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4     candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5     EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6     writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7     CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8     measured by the shipped experiment harness; person features enter only as feature
columns the
9     learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11     Run (on the GPU box):
12         python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14     Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15     `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16     """
17
18     from __future__ import annotations
19
20     import argparse
21     import json
22     import os
23     import time
24     from typing import Any, Dict, List, Optional, Sequence
25
26     from backend.eval import distill_local
27     from backend.eval.calibrate_wasb import load_golden_gts
28     from backend.eval.golden_manifest import load_resolved_manifest
29     from backend.eval.training import label_candidates
30     from backend.pipeline.detectors import fusion_features as ff
31     from backend.pipeline.detectors import rally_gate
32     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
33     from backend.utils.run_telemetry import RunTelemetry
34
35     # The 5 fps/resolution-invariant shuttle features distill_local already computes.
36     SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
37     # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
38     PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
39     SOURCE_MANIFEST_ENV = "RALLY_GOLDEN_SOURCE_MANIFEST"
40
41
42     def is_full_fusion_feature_file(path: str) -> bool:

```

```

43     """True when an existing golden feature JSON already has shuttle+person cue dicts.
44
45     Corpus-growth runs can leave leaner windowing-only JSONs with the same filename
46     pattern.
47     Those are useful for gts/windowing analysis, but they are not the replayable fusion
48     substrate that ``fusion_compare`` and ``ablation`` need.
49     """
50     try:
51         with open(path, encoding="utf-8") as f:
52             data = json.load(f)
53     except (OSError, json.JSONDecodeError, TypeError):
54         return False
55     candidates = data.get("candidates")
56     if not isinstance(candidates, list):
57         return False
58     if not candidates:
59         return True
60     return all(
61         isinstance(candidate, dict)
62         and "shuttle" in candidate
63         and "person" in candidate
64         for candidate in candidates
65     )
66
67
68 def load_source_video_paths(source_manifest_path: Optional[str]) -> Dict[str, str]:
69     """Load ``video_id -> local_path`` from the collector golden-source manifest.
70
71     The collector records source/proxy ownership and full-file MD5s. Only entries marked
72     ``present_local`` with a concrete ``local_path`` can drive local GPU extraction.
73     """
74
75     if not source_manifest_path:
76         return {}
77     with open(source_manifest_path, encoding="utf-8") as f:
78         data = json.load(f)
79     entries = data.get("videos", data) if isinstance(data, dict) else data
80     if not isinstance(entries, list):
81         raise ValueError(f"source manifest must contain a videos list: {source_manifest_path}")
82
83     paths: Dict[str, str] = {}
84     for entry in entries:
85         if not isinstance(entry, dict):
86             continue
87         video_id = entry.get("video_id") or entry.get("name")
88         local_path = entry.get("local_path")
89         if not video_id or not local_path:
90             continue
91         if entry.get("source_status", "present_local") != "present_local":
92             continue
93         paths[str(video_id)] = str(local_path)
94     return paths
95
96
97 def attach_source_video_paths(
98     specs: Sequence[Dict[str, Any]], source_manifest_path: Optional[str]
99 ) -> int:
100     """Attach explicit ``video_path`` fields to manifest specs from the source manifest.
101
102     Returns the number of specs updated. The collector source manifest is authoritative
103     for local source/proxy video location and overrides derived/stale manifest paths.
104     """
105

```

```

106     source_paths = load_source_video_paths(source_manifest_path)
107     updated = 0
108     for spec in specs:
109         name = str(spec.get("name", ""))
110         source_path = source_paths.get(name)
111         if not source_path:
112             continue
113         if spec.get("video_path") == source_path:
114             continue
115         spec["video_path"] = source_path
116         updated += 1
117     return updated
118
119
120     def _filter_specs(
121         specs: Sequence[Dict[str, Any]], only: Sequence[str]
122     ) -> List[Dict[str, Any]]:
123         if not only:
124             return list(specs)
125         wanted = list(dict.fromkeys(only))
126         wanted_set = set(wanted)
127         filtered = [spec for spec in specs if str(spec.get("name")) in wanted_set]
128         found = {str(spec.get("name")) for spec in filtered}
129         missing = [name for name in wanted if name not in found]
130         if missing:
131             raise ValueError(f"--only requested unknown video(s): {' '.join(missing)}")
132         return filtered
133
134
135     def _derive_video_path(spec: Dict[str, Any]) -> str:
136         """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
137         golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
138         convention)."""
139         if spec.get("video_path"):
140             return str(spec["video_path"])
141         labels = str(spec["labels"])
142         if labels.endswith(".rallies.csv"):
143             return labels[: -len(".rallies.csv")] + ".MP4"
144         raise ValueError(
145             f"{spec.get('name')}: no video_path and labels {labels!r} isn't *.rallies.csv"
146         )
147
148     def extract_video(
149         spec: Dict[str, Any],
150         detector: Optional[Any] = None,
151         frame_skip: int = 3,
152         iou: float = 0.5,
153         log_every_sec: float = 15.0,
154         telemetry: Optional[Any] = None,
155         use_gpu: bool = True,
156         detector_model: Optional[str] = None,
157         max_gpu_temp_c: Optional[float] = None,
158     ) -> Dict[str, Any]:
159         """One golden video -> a feature record (shuttle + person features per candidate
160         window).
161
162         Returns `{name, fps, frame_width,
163         candidates:[{start,end,shuttle{ },person{ },label}], gts}`.
164         `detector` (anything with `detections_over_windows`) is injectable for GPU-free
165         tests;
166         when None a real `PersonDetector` is built (loads torchvision on first use).
167         `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health
168         snapshots
169         from the detection progress callback so a hard kill leaves a forensic runlog.

```

```

166     """
167     fps, fw = float(spec["fps"]), float(spec["frame_width"])
168     traj = str(spec["trajectory"])
169     points = TrackNetRunner.parse_trajectory_csv(traj)
170     intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
171     gts = load_golden_gts(str(spec["labels"]))
172     labels = label_candidates(intervals, gts, iou)
173     # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
estimate
174     # (parity with the shuttle net-crossing gate) rather than hard-coding an
orientation.
175     axis, net_px, _span = rally_gate.estimate_play_axis(points)
176     video_path = _derive_video_path(spec)
177     if not os.path.isfile(video_path):
178         raise FileNotFoundError(
179             f"{spec.get('name')}: source video not found for fusion extraction:
{video_path}"
180         )
181
182     if detector is None:
183         from backend.pipeline.detectors.person_detector import PersonDetector
184
185         indexing_cfg: Dict[str, Any] = {"use_gpu": use_gpu}
186         if detector_model:
187             indexing_cfg["detector_model"] = detector_model
188         detector = PersonDetector({"indexing": indexing_cfg})
189
190     # Single-pass person detection over ALL candidate windows (open the video ONCE, read
191     # forward ? no per-window seek). Then compute features per window (pure, no GPU).
192     n = len(intervals)
193     win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
194     print(
195         f"[fusion_golden]   {spec['name']}: {n} windows ? person-detecting in one
pass...",
196         flush=True,
197     )
198     t0 = time.monotonic()
199     state = {"last": t0}
200
201     def _progress(done: int, total: int) -> None:
202         rec = None
203         if telemetry is not None:
204             rec = telemetry.snapshot(done, total, phase=spec["name"])
205         if max_gpu_temp_c is not None and rec is not None:
206             gpu = rec.get("gpu") or {}
207             temp = gpu.get("temp_c")
208             if temp is not None and float(temp) >= max_gpu_temp_c:
209                 raise RuntimeError(
210                     f"{spec['name']}: GPU temperature {float(temp):.1f}C reached "
211                     f"--max-gpu-temp-c {max_gpu_temp_c:.1f}; aborting before thermal
shutdown"
212                 )
213         now = time.monotonic()
214         if now - state["last"] >= log_every_sec or done >= total:
215             el = now - t0
216             rate = done / el if el > 0 else 0.0
217             eta = (total - done) / rate if rate > 0 else 0.0
218             pct = 100 * done // total if total else 0
219             print(
220                 f"[fusion_golden]   {spec['name']}: detect frame {done}/{total} ({pct}%)
"
221                 f"| {el:.0f}s elapsed | ETA {eta:.0f}s",
222                 flush=True,
223             )
224             state["last"] = now

```



```

225
226 per_window = detector.detections_over_windows(
227     video_path, win_frames, frame_skip, progress_cb=_progress
228 )
229
230 candidates: List[Dict[str, Any]] = []
231 for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
232     det = per_window[i]
233     det_fw = det.get("frame_width") or fw
234     det_fh = det.get("frame_height") or 0.0
235     # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
236     # normalisation fusion_features uses (x/width, y/height).
237     if axis == "x":
238         net_norm = net_px / det_fw if det_fw > 0 else 0.5
239     else:
240         net_norm = net_px / det_fh if det_fh > 0 else 0.5
241     sf, ef = win_frames[i]
242     shuttle_in = [p for p in points if sf <= p.frame <= ef]
243     person = ff.compute_person_features(
244         det["frames"],
245         shuttle_in,
246         det_fw,
247         det_fh,
248         court_polygon=None,
249         net=(axis, net_norm),
250     )
251     candidates.append(
252         {
253             "start": float(s),
254             "end": float(e),
255             "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
256             "person": {k: float(v) for k, v in person.items()},
257             "label": int(labels[i]),
258         }
259     )
260     return {
261         "name": spec["name"],
262         "fps": fps,
263         "frame_width": fw,
264         "candidates": candidates,
265         "gts": [[float(a), float(b)] for a, b in gts],
266     }
267
268
269 def run(
270     manifest_path: str,
271     out_dir: str,
272     frame_skip: int = 3,
273     iou: float = 0.5,
274     fresh: bool = False,
275     smallest_first: bool = False,
276     source_manifest: Optional[str] = None,
277     only: Sequence[str] = (),
278     missing_fusion_only: bool = False,
279     use_gpu: bool = True,
280     detector_model: Optional[str] = None,
281     max_gpu_temp_c: Optional[float] = None,
282 ) -> List[str]:
283     specs = load_resolved_manifest(manifest_path)
284     source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
285     if source_manifest:
286         updated = attach_source_video_paths(specs, source_manifest)
287         print(
288             f"[fusion_golden] attached {updated}/{len(specs)} source video path(s) "

```

```

289         f"from {source_manifest}",
290         flush=True,
291     )
292     specs = _filter_specs(specs, only)
293     if smallest_first:
294         # Process small videos first for quick feedback (the slow full match lands
last);
295         # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
296         specs = sorted(
297             specs,
298             key=lambda s: (
299                 os.path.getsize(s["trajectory"])
300                 if os.path.isfile(s.get("trajectory", ""))
301                 else 0
302             ),
303         )
304     os.makedirs(out_dir, exist_ok=True)
305     # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
keep a tiny
306     # rotating runlog of velocity + machine health for the post-mortem. Constructing it
must never
307     # break extraction, so guard the build and fall back to no telemetry on any error.
308     telem: Optional[RunTelemetry]
309     try:
310         telem = RunTelemetry(
311             os.path.join(out_dir, "run_telemetry.jsonl"), label="fusion_golden"
312         )
313     except Exception: # noqa: BLE001 - telemetry wiring must never break the run it
observes
314         telem = None
315     written: List[str] = []
316     for vi, spec in enumerate(specs, 1):
317         out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
318         existing = os.path.isfile(out)
319         existing_full = existing and is_full_fusion_feature_file(out)
320         # Resumable: skip a video whose full-fusion JSON already exists. A leaner
321         # windowing-only JSON is stale for A7, so re-extract it even without --fresh.
322         if missing_fusion_only and not fresh and existing_full:
323             print(
324                 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: full-fusion "
325                 f"features already present -> {out}",
326                 flush=True,
327             )
328             written.append(out)
329             continue
330         if not fresh and existing and existing_full:
331             print(
332                 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already extracted
"
333                 f"-> {out} (use --fresh to redo)",
334                 flush=True,
335             )
336             written.append(out)
337             continue
338         if not fresh and existing and not existing_full:
339             print(
340                 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: existing JSON "
341                 f"is not full-fusion schema; re-extracting...",
342                 flush=True,
343             )
344             print(
345                 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
346                 flush=True,
347             )

```

```

348         t0 = time.monotonic()
349         if telem is not None:
350             telem.event("video_start", spec["name"])
351         try:
352             data = extract_video(
353                 spec,
354                 frame_skip=frame_skip,
355                 iou=iou,
356                 telemetry=telem,
357                 use_gpu=use_gpu,
358                 detector_model=detector_model,
359                 max_gpu_temp_c=max_gpu_temp_c,
360             )
361         except Exception as e: # noqa: BLE001 - record machine state at death, then re-
raise
362             if telem is not None:
363                 telem.event("error", spec["name"], err=str(e))
364             raise
365         n_pos = sum(c["label"] for c in data["candidates"])
366         with open(out, "w", encoding="utf-8") as f:
367             json.dump(data, f, indent=2)
368         if telem is not None:
369             telem.event(
370                 "video_done",
371                 spec["name"],
372                 windows=len(data["candidates"]),
373                 positives=n_pos,
374             )
375         print(
376             f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
377             f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
378             f"-> {out}",
379             flush=True,
380         )
381         written.append(out)
382     return written
383
384
385     def main() -> None:
386         ap = argparse.ArgumentParser(
387             description="Extract golden-set shuttle+person features (fusion P3 GPU step)."
388         )
389         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
390         ap.add_argument("--out-dir", default="output")
391         ap.add_argument("--frame-skip", type=int, default=3)
392         ap.add_argument("--iou", type=float, default=0.5)
393         ap.add_argument(
394             "--fresh", action="store_true", help="re-extract even if a JSON already exists"
395         )
396         ap.add_argument(
397             "--smallest-first",
398             action="store_true",
399             help="process smaller videos first for quick feedback (LOVO is order-
independent)",
400         )
401         ap.add_argument(
402             "--source-manifest",
403             default=None,
404             help=(
405                 "collector golden_source_videos.json; overrides video paths for source/proxy
"
406                 f"videos (or set {SOURCE_MANIFEST_ENV})"
407             ),

```

```

408     )
409     ap.add_argument(
410         "--only",
411         action="append",
412         default=[],
413         metavar="VIDEO_ID",
414         help="process one manifest video by name; repeatable",
415     )
416     ap.add_argument(
417         "--missing-fusion-only",
418         action="store_true",
419         help="skip videos whose existing JSON already has full shuttle+person fusion
schema",
420     )
421     ap.add_argument(
422         "--cpu",
423         action="store_true",
424         help="run torchvision person detection on CPU instead of CUDA",
425     )
426     ap.add_argument(
427         "--detector-model",
428         default=None,
429         choices=[
430             "fasterrcnn_resnet50_fpn",
431             "fasterrcnn_mobilenet_v3_large_fpn",
432             "retinanet_resnet50_fpn",
433         ],
434         help="torchvision detector model; default keeps the configured Faster R-CNN
ResNet",
435     )
436     ap.add_argument(
437         "--max-gpu-temp-c",
438         type=float,
439         default=None,
440         help="abort extraction if telemetry sees GPU temperature at or above this
value",
441     )
442     args = ap.parse_args()
443     run(
444         args.manifest,
445         args.out_dir,
446         args.frame_skip,
447         args.iou,
448         fresh=args.fresh,
449         smallest_first=args.smallest_first,
450         source_manifest=args.source_manifest,
451         only=args.only,
452         missing_fusion_only=args.missing_fusion_only,
453         use_gpu=not args.cpu,
454         detector_model=args.detector_model,
455         max_gpu_temp_c=args.max_gpu_temp_c,
456     )
457
458
459     if __name__ == "__main__":
460         main()
461

```

7. user (2026-07-04T00:44:51.605Z)

```

1     """Fusion P4 ? measure the INCREMENTAL audio-cue lift on top of the person cue (CPU-
only).
2
3     Reuses the person-feature JSONs from `fusion_golden.py` (no GPU re-detect): `augment`
runs

```

```

4      `detect_hits_in_video` once per video (ffmpeg + numpy, CPU) and adds the 3 audio
features per
5      window; `compare_increment` then runs the SHIPPED harness between a shuttle+person
variant
6      and a shuttle+person+audio variant ? so audio is measured as a marginal contribution ON
TOP
7      of person, not against raw shuttle. Audio enters only as columns the learned scorer
weights
8      (no special-casing, default-OFF); the golden-set ?F1 decides whether it earns its keep.
9
10     Run (no GPU):
11         python -m backend.eval.fusion_audio --manifest output/human_lovo_manifest.json
--features-dir output --augment --compare
12         ""
13
14     from __future__ import annotations
15
16     import argparse
17     import json
18     import logging
19     import os
20     from typing import Any, Dict, List, Optional, Sequence
21
22     from backend.eval import experiment
23     from backend.eval.fusion_compare import delta_stats
24     from backend.eval.fusion_golden import (
25         PERSON_FEATURE_NAMES,
26         SHUTTLE_FEATURE_NAMES,
27         SOURCE_MANIFEST_ENV,
28         _derive_video_path,
29         _filter_specs,
30         attach_source_video_paths,
31     )
32     from backend.eval.golden_manifest import load_resolved_manifest
33     from backend.pipeline.audio.hit_detector import detect_hits_in_video
34     from backend.pipeline.detectors import audio_features as af
35
36     logger = logging.getLogger(__name__)
37
38
39     def augment(
40         manifest_path: str,
41         features_dir: str,
42         k: float = 2.5,
43         source_manifest: Optional[str] = None,
44         only: Sequence[str] = (),
45     ) -> List[str]:
46         """Add the 3 audio features to each existing ``<name>_golden_features.json``
window."""
47         specs = load_resolved_manifest(manifest_path)
48         source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
49         if source_manifest:
50             updated = attach_source_video_paths(specs, source_manifest)
51             print(
52                 f"[fusion_audio] attached {updated}/{len(specs)} source video path(s) "
53                 f"from {source_manifest}",
54                 flush=True,
55             )
56         specs = _filter_specs(specs, only)
57         written: List[str] = []
58         for spec in specs:
59             jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
60             if not os.path.isfile(jp):
61                 print(
62                     f"[fusion_audio] {spec['name']}: no feature JSON ? run fusion_golden

```

```

first; skipping"
63         )
64         continue
65         video = _derive_video_path(spec)
66         if not os.path.isfile(video):
67             raise FileNotFoundError(
68                 f"{spec.get('name')}: source video not found for audio augmentation:
{video}"
69             )
70         print(
71             f"[fusion_audio] {spec['name']}: detecting audio hits (k={k})...",
72             flush=True,
73         )
74         hits = detect_hits_in_video(video, k=k)
75         with open(jp, encoding="utf-8") as f:
76             data = json.load(f)
77             for c in data["candidates"]:
78                 c["audio"] = af.compute_audio_features(hits, c["start"], c["end"])
79             with open(jp, "w", encoding="utf-8") as f:
80                 json.dump(data, f, indent=2)
81             print(
82                 f"[fusion_audio] {spec['name']}: {len(hits)} hits over
{len(data['candidates'])} windows",
83                 flush=True,
84             )
85             written.append(jp)
86         return written
87
88
89     def load_videos_with_audio(features_dir: str) -> List[experiment.VideoCandidates]:
90         """Load the FULL-FUSION-SCHEMA feature JSONs into VideoCandidates including the
audio
91         columns (requires `augment` to have run ? raises if a fusion JSON lacks audio).
92
93         Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts) are
SKIPPED, not
94         crashed on, so a mixed corpus loads the fusion subset cleanly (see
``_is_fusion_schema``).
95         The skip is checked BEFORE the audio-presence guard, so a genuine fusion file that
simply
96         hasn't been ``--augment``-ed still raises rather than being silently dropped."""
97         return experiment.load_golden_videos(
98             features_dir,
99             shuttle_columns=SHUTTLE_FEATURE_NAMES,
100             person_columns=PERSON_FEATURE_NAMES,
101             audio_columns=af.AUDIO_FEATURE_NAMES,
102             audio_policy="require",
103             skip_logger=logger,
104             skip_message="fusion_audio.load_videos_with_audio: skipping %s ? windowing-only
"
105             "schema (no shuttle/person cue dicts), not a fusion-feature file",
106             no_audio_error="{path} has no audio features ? run ``--augment`` first",
107         )
108
109
110     def compare_incremental(
111         videos: List[experiment.VideoCandidates], iou: float = 0.5, threshold: float = 0.5
112     ) -> Dict[str, Any]:
113         """Incremental audio lift: shuttle+person vs shuttle+person+audio (honest
LOVO)."""
114         person = experiment.Variant(
115             name="shuttle_person",
116             feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES),
117             threshold=threshold,
118         )

```

```

119     person_audio = experiment.Variant(
120         name="shuttle_person_audio",
121         feature_names=list(SHUTTLE_FEATURE_NAMES)
122         + list(PERSON_FEATURE_NAMES)
123         + list(af.AUDIO_FEATURE_NAMES),
124         threshold=threshold,
125     )
126     return experiment.compare(videos, person, person_audio, iou=iou, honest=True)
127
128
129     def format_result(res: Dict[str, Any], iou: float) -> str:
130         a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
131         ds = delta_stats(res)
132         st = ds["sign_test"]
133         lines = [
134             f"=== Fusion P4 incremental AUDIO LOVO (n={res['num_videos']}, IoU>={iou},
135             honest) ===",
136             f"  shuttle+person          : F1={a['f1']:.3f}  P={a['precision']:.3f}
137             R={a['recall']:.3f}",
138             f"  shuttle+person+audio : F1={b['f1']:.3f}  P={b['precision']:.3f}
139             R={b['recall']:.3f}",
140             f"  incremental ?F1 = {d['f1']:.3f}  ({res['delta_pct']['f1']:.1f}%)",
141             f"  paired ?F1 {ds['mean_delta']:.3f}  [95% CI {ds['ci_low']:.3f},
142             {ds['ci_high']:.3f}]  "
143             f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
144             f"| CI clears 0: {ds['ci_excludes_zero']}",
145         ]
146         for pv in res["per_video_delta"]:
147             lines.append(f"    {pv['video']}<26s} ?F1={pv['f1']:.3f}")
148         return "\n".join(lines)
149
150     def main() -> None:
151         ap = argparse.ArgumentParser(
152             description="Fusion P4 ? incremental audio lift on the golden set."
153         )
154         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
155         ap.add_argument("--features-dir", default="output")
156         ap.add_argument("--k", type=float, default=2.5, help="hit-detector sensitivity")
157         ap.add_argument(
158             "--source-manifest",
159             default=None,
160             help=(
161                 "collector golden_source_videos.json; overrides video paths for source/proxy
162                 "
163                 f"videos (or set {SOURCE_MANIFEST_ENV})"
164             ),
165         )
166         ap.add_argument(
167             "--only",
168             action="append",
169             default=[],
170             metavar="VIDEO_ID",
171             help="augment one manifest video by name; repeatable",
172         )
173         ap.add_argument(
174             "--augment",
175             action="store_true",
176             help="add audio features to the existing JSONs",
177         )
178         ap.add_argument(
179             "--compare", action="store_true", help="report the incremental audio LOVO ?F1"
180         )
181         ap.add_argument("--iou", type=float, default=0.5)
182         ap.add_argument("--record", action="store_true")

```

```

179         args = ap.parse_args()
180
181     if args.augment:
182         augment(
183             args.manifest,
184             args.features_dir,
185             args.k,
186             source_manifest=args.source_manifest,
187             only=args.only,
188         )
189     if args.compare:
190         videos = load_videos_with_audio(args.features_dir)
191         if len(videos) < 2:
192             raise SystemExit(f"need >=2 augmented feature files in {args.features_dir}")
193         res = compare_increment(videos, iou=args.iou)
194         print(format_result(res, args.iou))
195         if args.record:
196             row = experiment.record_experiment(res)
197             print(
198                 f"recorded -> eval_baselines/experiment_leaderboard.json
199                 ({row['timestamp']})"
200             )
201
202     if __name__ == "__main__":
203         main()
204

```

8. assistant (2026-07-04T00:45:10.483Z)

Now let me analyze the three specific angles you mentioned:

Analysis

Based on my review of the PR diff and the two files, here are the simplification candidates:

1. `--missing-fusion-only` flag analysis (Lines 322-343 in fusion_golden.py)

The flag logic has three branches:

- Line 322-329: `if missing\_fusion\_only and not fresh and existing\_full` ? skip with "full-fusion features already present"
- Line 330-337: `if not fresh and existing and existing\_full` ? skip with "already extracted"
- Line 338-343: `if not fresh and existing and not existing\_full` ? print "re-extracting"

****The problem:**** When `existing\_full=True`, BOTH the first and second conditions skip the video. The first condition only DIFFERS in the log message. When `existing\_full=True` and `missing\_fusion\_only=False` (the default), the second condition skips. When `missing\_fusion\_only=True`, the first condition skips first. The flag changes ONLY which log message prints? it doesn't change execution behavior.

The default behavior already re-extracts windowing-only JSONs and skips full-fusion ones, so `--missing-fusion-only` is ****functionally a no-op**** that just picks a different message.

2. Duplicated source-manifest attach block (Lines 284-291 in fusion_golden.py and 48-55 in fusion_audio.py)

The exact same 6-line block appears in both files:

```

```python
source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
if source_manifest:
 updated = attach_source_video_paths(specs, source_manifest)
 print(
 f"[fusion_{module}] attached {updated}/{len(specs)} source video path(s) "
 f"from {source_manifest}",
 flush=True,

```



```
...)
```

This can be extracted into a ``_attach_and_log_source_manifest()`` helper in `fusion_golden.py` and imported by `fusion_audio.py`.

### 3. ``load_source_video_paths`` redundancy (Lines 68-94 in `fusion_golden.py`)

Line 79: ``entries = data.get("videos", data) if isinstance(data, dict) else data``

This handles two cases:

- If ``data`` is dict with key "videos", use ``data["videos"]``
- If ``data`` is dict without "videos", use ``data`` itself
- If ``data`` is not dict, use ``data``

Then line 80 checks ``if not isinstance(entries, list)`` and raises. This means the fallback path ``if isinstance(data, dict) else data`` can only work if the dict case has no "videos" key AND ``data`` itself is a list. But that's contradictory? if ``data`` is a dict, it can't also be a list. **\*\*This branch is unreachable dead code.\*\*** The line should be:

```
```python
entries = data.get("videos", data) if isinstance(data, dict) else data
```
```

But the ``if isinstance(data, dict) else data`` part only makes sense if we expect a bare list at the top level, yet the loop later checks ``if not isinstance(entry, dict)`` suggesting entries ARE expected to be dicts. The condition is over-protective.

### 4. ``_progress`` closure nesting (Lines 201-224 in `fusion_golden.py`)

The ``rec`` variable is only needed for the temperature check. The closure could be cleaner if the temperature check is factored out, or the ``rec`` assignment could happen unconditionally to avoid the nested condition.

Current:

```
```python
rec = None
if telemetry is not None:
    rec = telemetry.snapshot(done, total, phase=spec["name"])
if max_gpu_temp_c is not None and rec is not None:
    # temp check
```
```

Could be:

```
```python
if telemetry is not None:
    rec = telemetry.snapshot(done, total, phase=spec["name"])
    if max_gpu_temp_c is not None and rec is not None:
        # temp check
```
```

Or extract temp-check into a helper: ``_check_gpu_temp(rec, max_gpu_temp_c, spec)``.

JSON output with up to 6 candidates:

```
```json
[
  {
    "file": "backend/eval/fusion_golden.py",
    "line": "322-337",
    "summary": "--missing-fusion-only flag changes only log message, not behavior; when existing_full=True both conditions skip regardless; functionally a no-op flag masking as a control",
    "failure_scenario": "User runs --missing-fusion-only expecting a different re-extraction policy but gets identical skips; misleading CLI surface"
  },
  {

```

```

    "file": "backend/eval/fusion_golden.py & backend/eval/fusion_audio.py",
    "line": "284-291 & 48-55",
    "summary": "Duplicated 6-line source-manifest attach+log block in both modules; extract into
_attach_and_log_source_manifest() helper",
    "failure_scenario": "Maintenance burden: any change to attach/log logic requires edits in
two places; inconsistent message tweaks risk divergence"
  },
  {
    "file": "backend/eval/fusion_golden.py",
    "line": "79",
    "summary": "load_source_video_paths line 79 has unreachable fallback: 'data if
isinstance(data, dict) else data' ? if data is dict, data is never a list, so the else branch is
dead; should simplify to 'data.get(\"videos\", data) if isinstance(data, dict) else data' but
the else case (bare list) conflicts with later dict-entry checks",
    "failure_scenario": "Over-protective code muddies intent; if a bare list is not a valid
input, the isinstance check should reject it, not silently accept it"
  },
  {
    "file": "backend/eval/fusion_golden.py",
    "line": "201-212",
    "summary": "_progress closure initializes rec=None then assigns conditionally, then checks
'if max_gpu_temp_c is not None and rec is not None' separately; nesting the temp check inside
the telemetry block avoids the double-guard",
    "failure_scenario": "Readability: rec decoupled from its only use; reader must trace two
conditions to understand when temperature is checked; cleaner if factored into
_check_gpu_temp(rec, spec, threshold) helper"
  }
]
...

```